

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №5
по дисциплине «Параллельные алгоритмы»
Тема: Умножение матриц на GPU

Студент гр. 1304

Шаврин А.П.

Преподаватель

Сергеева Е.И.

Санкт-Петербург

2024

Цель работы.

Изучить параллельное блочное умножение матриц в стандарте OpenCL с учётом оптимизаций доступа к локальной и глобальной памяти.

Задание.

Реализовать блочное умножение матриц в стандарте OpenCL (или CUDA) с учётом оптимизаций доступа к локальной и глобальной памяти. Выполнить тестирование: сравнение результатов вычислений с полученными в работе 4. В отчете: Произвести сравнение производительности с CPU реализациями сделанными в лаб. работе 4

Выполнение работы.

1. Были написаны файлы *io.hpp* и *io.cpp* в которых определены функции:

void printPlatformInfo(cl_platform_id platform); - функция вывода информации о платформе

void printDeviceInfo(cl_device_id device); - функция вывода информации о девайсе

std::vector<int> generateMatrix(const int size, const int maxValue) – генерация матриц заданной размерности

void writeMatrixInFile(const std::string& filename, const std::vector<int>& result, const int size) - запись матрицы в файл

void printMatrix(const std::vector<int>& result, const int size) – вывод матрицы в консоль

2. После были созданы файлы *utils.hpp* и *utils.cpp* в которых описаны функции обертки для стандартных функций CL с целью обработки исключений:

cl_platform_id getPlatform(); - получение платформы

cl_device_id getDevice(cl_platform_id platform); - получение девайса

cl_context getContext(cl_device_id device); - получение контекста

cl_command_queue getQueue(cl_context context, cl_device_id device); -

получение очереди команд

void checkBufferCreateError(cl_int CL_err, const std::string& bufferName); -

проверка ошибок создания буфера

std::string loadKernel(const std::string &filename); - загрузка ядра из файла

cl_program getProgram(cl_context context, const std::string& kernelSource);

- создание программы

void buildProgram(cl_program program, cl_device_id device); - сборка

программы

cl_kernel getKernel(cl_program program, const std::string& task); -

получение ядра

void runKernel(

cl_kernel kernel,

cl_command_queue queue,

size_t localWorkSize[2],

size_t globalWorkSize[2]

); - запуск ядра с замером времени выполнения *void*

readResult(cl_command_queue queue, cl_mem bufferC, std::vector<int>& C); -

чтение результата из буфера.

3. Реализована функция `main` в файле `main.cpp` выполняющая всю программу

4. Написан файл `kernel.cl` по блочному умножению матрицы с использованием локальных буферов.

5. Сравнение производительности.

По таблице 1 видно, что для небольших матриц (до 32) время вычисления на хосте в стандарте OpenCL больше. Это связано с загрузкой данных и созданием контекста OpenCL. По сравнению с этим временем, время самого вычисления пренебрежимо мало.

Для больших матриц (от 32) время вычисления возрастает, но остаётся намного меньшим, чем время вычисления с помощью алгоритмов, использованных в 4 лабораторной работе. Матричные вычисления с помощью

OpenCL на GPU быстрее, потому что GPU обеспечивает параллельное выполнение на тысячи потоков, имеет быструю память и высокую пропускную способность, что делает его гораздо более эффективным для выполнения таких вычислений по сравнению с традиционными процессорами.

Результаты тестирования см. в приложении А.

Выводы.

В ходе работы было изучено и реализовано параллельное блочное умножение матриц в стандарте OpenCL с учётом оптимизаций доступа к локальной и глобальной памяти

В результате сравнения с алгоритмами из предыдущей работы было установлено, что вычисления на GPU с использованием стандарта OpenCL намного производительнее.

ПРИЛОЖЕНИЕ А

ТЕСТИРОВАНИЕ

Таблица 1 – Сравнение производительности алгоритмов умножения матриц в зависимости от размеров матриц. (К – kernel, Н - host)

№ п/п	Размеры матриц	Время выполнения в мс для блочного алгоритма	Время выполнения в мс для алгоритма Штрассена	Время выполнения блочного алгоритма на GPU в стандарте OpenCL
1.	32	0.6922	0.1758	K=0.04468 H=2.7547
2.	128	9.6360	11.3526	K= 0.1842 H= 2.5925
3.	256	66.1122	76.0608	K= 0.5398 H= 3.3327
4.	512	651.008	537.968	K= 2.38828 H= 5.6457
5.	1024	6693.79	3822.23	K= 17.7694 H= 21.0092
6.	2048	146055	27156	K= 130.508 H= 136.493